

Delta Lake & Medallion Architecture Guide

ACID lakehouse tiering, Change Data Feed, and table maintenance

How Delta Lake gives a data lake transactional guarantees, and how to organise tables into Bronze, Silver and Gold tiers that each have a clear contract. Covers the transaction log, MERGE and upsert patterns, Change Data Feed for incremental downstream sync, schema evolution, and the maintenance jobs (OPTIMIZE, Z-ORDER, VACUUM) that keep a lakehouse fast.

Vivek Pandey — Data Engineer
Azure · Databricks · PySpark · Delta Lake
vp171097.github.io/DevWebsite

Contents

1. Why Delta: the problem with plain Parquet
2. The transaction log and ACID guarantees
3. The medallion architecture
4. Bronze: raw, append-only ingestion
5. Silver: cleansed, conformed, deduplicated
6. Gold: business aggregates and dimensions
7. MERGE, upserts and SCD Type 2
8. Change Data Feed (CDF)
9. Schema evolution vs schema enforcement
10. OPTIMIZE, Z-ORDER and liquid clustering
11. VACUUM, time travel and retention
12. Maintenance checklist

1. Why Delta: the problem with plain Parquet

A directory of Parquet files is a good storage format and a poor table. It has no atomic commit, so a reader can see a half-written job. It has no schema enforcement, so one bad writer corrupts the dataset. It has no update or delete, so a GDPR erasure request means rewriting partitions by hand. And listing millions of files to plan a query is slow.

Delta Lake keeps the Parquet files and adds a transaction log beside them. That single addition supplies atomicity, schema enforcement, updates and deletes, time travel, and fast metadata.

Capability	Plain Parquet	Delta Lake
Atomic writes	No — partial output visible to readers	Yes — commit is a single atomic log entry
UPDATE / DELETE / MERGE	Manual partition rewrite	Supported as SQL/DataFrame operations
Schema enforcement	None	Rejects incompatible writes by default
Time travel	No	Query any prior version by number or timestamp
Concurrent writers	Unsafe	Optimistic concurrency control
Metadata/planning	File listing	Log checkpoint with per-file statistics

2. The transaction log and ACID guarantees

The log lives in **_delta_log/** beside the data. Each commit writes one JSON file containing the actions that make up that transaction — files added, files removed, metadata changes. The table's current state is the result of replaying those actions in order.

```
my_table/
■■■ part-00000-....snappy.parquet
■■■ part-00001-....snappy.parquet
■■■ _delta_log/
    ■■■ 00000000000000000000000000000000.json    <- commit 0: CREATE + first files
    ■■■ 00000000000000000000000000000001.json    <- commit 1: add/remove actions
    ■■■ ...
    ■■■ 00000000000000000000000000000010.checkpoint.parquet    <- every 10 commits
```

Checkpoints collapse the log so readers do not replay every commit from zero.

A commit entry records adds and removes rather than mutating files. An UPDATE therefore writes new files and marks the old ones removed; the old files remain on storage until VACUUM deletes them, which is exactly what makes time travel possible.

```
-- Inspect the history and what each operation did
DESCRIBE HISTORY portfolio.ecom_silver.orders;

-- Per-file statistics the optimizer uses for data skipping
SELECT * FROM json.`/mnt/lake/orders/_delta_log/00000000000000000000000000000001.json`;
```

Optimistic concurrency Two writers may proceed simultaneously; at commit time each checks whether the files it read were changed. A conflict raises **ConcurrentModificationException** rather than silently corrupting data. Append-only writers rarely conflict; blind overwrites of the same partition frequently do.

3. The medallion architecture

Medallion is a naming convention for a pipeline with three contracts. Its value is not the colours — it is that each layer has one job, so a failure has one obvious owner.

Layer	Contract	Typical operations	Who reads it
Bronze	Exactly what the source sent, never edited	Append raw records + ingestion metadata	Data engineers, replay/audit
Silver	Clean, typed, deduplicated, conformed	Cast, validate, dedupe, quarantine, join reference data	Engineers, analysts, ML
Gold	Business definitions, ready to serve	Aggregate, model as facts/dimensions	BI, dashboards, executives

The rule that makes it work Bronze is immutable. When Silver logic has a bug you fix the code and reprocess from Bronze. If Bronze has been edited, that recovery path is gone and you are back to asking the source system to resend.

4. Bronze: raw, append-only ingestion

Bronze preserves the source payload and adds provenance columns — where the record came from, which file, when it arrived. Keep types permissive here; a malformed value should land in Bronze and be rejected in Silver, not crash ingestion.

```
from pyspark.sql import functions as F

bronze = (spark.readStream
    .format("cloudFiles") # Databricks Auto Loader
    .option("cloudFiles.format", "csv")
    .option("cloudFiles.schemaLocation", f"{CHECKPOINT}/schema")
    .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
    .option("rescuedDataColumn", "_rescued_data") # never silently drop a field
    .load(LANDING_PATH)
    .select("?",
        F.current_timestamp().alias("_ingested_at"),
        F.input_file_name().alias("_source_file")))

(bronze.writeStream
    .format("delta")
    .option("checkpointLocation", f"{CHECKPOINT}/bronze_orders")
    .outputMode("append")
    .trigger(availableNow=True)
    .toTable("portfolio.ecom_bronze.orders"))
```

rescuedDataColumn captures fields that did not fit the schema instead of discarding them.

5. Silver: cleansed, conformed, deduplicated

Silver is where data quality is enforced. Two decisions define the layer: what counts as a duplicate, and what happens to a row that fails validation.

Deduplication on a business key

```
from pyspark.sql import Window

# Keep the most recent version of each order_id.
w = Window.partitionBy("order_id").orderBy(F.col("_ingested_at").desc())

silver = (bronze
    .withColumn("_rn", F.row_number().over(w))
    .filter("_rn = 1").drop("_rn")
    .withColumn("purchased_at", F.to_timestamp("order_purchase_timestamp"))
    .withColumn("price", F.col("price").cast("decimal(12,2)")))

# For streaming, bound state with a watermark instead of a window:
deduped = (bronze.withWatermark("_ingested_at", "24 hours")
    .dropDuplicates(["order_id", "_ingested_at"]))
```

Quarantine rather than drop

Silently dropping bad rows makes reconciliation impossible. Route failures to a quarantine table with the same schema plus the reason, so counts still add up.

```
RULES = {
    "valid_price": F.col("price") >= 0,
    "valid_status": F.col("status").isin("created", "shipped", "delivered", "cancelled"),
    "has_customer": F.col("customer_id").isNotNull(),
}
passes = F.lit(True)
for cond in RULES.values():
    passes = passes & cond

clean = silver.filter(passes)
bad = (silver.filter(~passes)
    .withColumn("_failed_rules",
        F.array_except(
            F.array([F.lit(n) for n in RULES]),
            F.array([F.when(c, F.lit(n)) for n, c in RULES.items()]))))

# Reconciliation must hold: bronze = silver + quarantine
assert bronze.count() == clean.count() + bad.count()
```

6. Gold: business aggregates and dimensions

Gold encodes definitions the business agrees on — what 'active customer' or 'net revenue' means. Model it as facts and dimensions so BI tools can navigate it without bespoke SQL.

```
agg_daily_sales = (orders.filter("status = 'delivered'")
    .join(order_items, "order_id")
    .groupBy(F.to_date("purchased_at").alias("order_date"))
    .agg(F.round(F.sum("price"), 2).alias("revenue"),
        F.countDistinct("order_id").alias("orders"),
        F.round(F.avg(F.col("delivered_at") > F.col("estimated_at"))
            .cast("double"), 4).alias("late_delivery_rate")))
```

Never coerce nulls to zero An order with no review has a null review score, not a score of 0. Turning it into 0 silently drags every average down and the error is almost impossible to find later.

7. MERGE, upserts and SCD Type 2

MERGE is the workhorse of incremental loading: insert new rows, update changed ones, optionally delete removed ones, in a single atomic commit.

```
MERGE INTO portfolio.ecom_silver.orders AS t
USING updates AS s
  ON t.order_id = s.order_id
WHEN MATCHED AND s._deleted = true THEN DELETE
WHEN MATCHED AND s.updated_at > t.updated_at THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

MERGE performance The ON condition decides how many files must be rewritten. Add a partition predicate so Delta can skip untouched partitions: **ON t.order_id = s.order_id AND t.order_date >= current_date() - INTERVAL 7 DAYS**. Without it, a small update can rewrite the whole table.

Slowly Changing Dimension Type 2

SCD2 keeps history: rather than overwriting an attribute, close the current row and open a new one, so facts can be joined to the attribute values that were true at the time.

```
-- Declarative form (Delta Live Tables / Lakeflow)
CREATE OR REFRESH STREAMING TABLE dim_customer;

APPLY CHANGES INTO live.dim_customer
FROM STREAM(live.customer_changes)
  KEYS (customer_unique_id)
  SEQUENCE BY change_ts
  STORED AS SCD TYPE 2
  TRACK HISTORY ON (zip_code_prefix, city, state);

-- Current rows are the ones still open:
SELECT * FROM dim_customer WHERE __END_AT IS NULL;
```

```
-- Manual SCD2 with MERGE: close the old version, then insert the new one.
MERGE INTO dim_customer t
USING changes s ON t.customer_id = s.customer_id AND t.is_current = true
WHEN MATCHED AND (t.city <> s.city OR t.state <> s.state) THEN
  UPDATE SET t.is_current = false, t.valid_to = s.change_ts;

INSERT INTO dim_customer
SELECT customer_id, city, state, change_ts AS valid_from,
       CAST(NULL AS TIMESTAMP) AS valid_to, true AS is_current
FROM changes;
```

8. Change Data Feed (CDF)

CDF makes a Delta table emit the row-level changes it underwent, so a downstream job can process only what changed instead of rescanning the table. It is the cleanest way to propagate Silver updates into Gold incrementally.

```
ALTER TABLE portfolio.ecom_silver.orders
  SET TBLPROPERTIES (delta.enableChangeDataFeed = true);

-- Changes are available from the version where the property was enabled.
SELECT * FROM table_changes('portfolio.ecom_silver.orders', 12, 18);
```

_change_type	Meaning
insert	New row
delete	Row removed
update_preimage	Row as it was before the update
update_postimage	Row as it is after the update

```
# Streaming consumption: only post-update and insert rows feed the aggregate.
changes = (spark.readStream.format("delta")
  .option("readChangeFeed", "true")
  .option("startingVersion", 12)
  .table("portfolio.ecom_silver.orders")
  .filter(F.col("_change_type").isin("insert", "update_postimage")))
```

Retention caveat CDF data is stored alongside the table and is removed by VACUUM according to your retention setting. A consumer that falls further behind than the retention window cannot catch up incrementally and must do a full reload.

9. Schema evolution vs schema enforcement

Enforcement is the default and it is a feature: a write whose schema does not match is rejected rather than corrupting the table. Evolution is the deliberate exception.

```
# Add newly-arrived columns on purpose
(df.write.format("delta").mode("append")
  .option("mergeSchema", "true")
  .saveAsTable("portfolio.ecom_bronze.orders"))

# Replace the schema wholesale (destructive – reprocessing scenarios only)
(df.write.format("delta").mode("overwrite")
  .option("overwriteSchema", "true")
  .saveAsTable("portfolio.ecom_bronze.orders"))
```

Change	Allowed with mergeSchema?	Notes
Add a column	Yes	Existing rows read as NULL
Widen type (int -> long)	Yes	Safe upcast
Narrow type (long -> int)	No	Would lose data; requires overwriteSchema
Rename a column	No	Enable column mapping, then ALTER ... RENAME COLUMN
Drop a column	No	Requires column mapping or a rewrite

10. OPTIMIZE, Z-ORDER and liquid clustering

Streaming and frequent small writes produce many small files. Each file carries planning and open cost, so query time drifts upward until compaction runs.

```
-- Compact into ~1 GB files, limited to the partitions that changed
OPTIMIZE portfolio.ecom_silver.orders
  WHERE order_date >= current_date() - INTERVAL 7 DAYS;

-- Co-locate related values so data skipping can prune more files
OPTIMIZE portfolio.ecom_silver.orders
  ZORDER BY (customer_id, product_id);
```

Z-ORDER sorts data by a space-filling curve over the chosen columns, clustering similar values into the same files. Choose columns that appear in high-selectivity filters; each extra column dilutes the benefit, so two or three is the practical limit.

Liquid clustering

Liquid clustering replaces both partitioning and Z-ORDER on newer Databricks runtimes. Clustering keys can be changed without rewriting the table, and it handles high-cardinality keys that would be a poor partition choice.

```
CREATE TABLE portfolio.ecom_silver.orders
  CLUSTER BY (order_date, customer_id);

-- Keys can be changed later; new data honours the new layout
ALTER TABLE portfolio.ecom_silver.orders CLUSTER BY (customer_id);
OPTIMIZE portfolio.ecom_silver.orders;
```

Technique	Cardinality suited to	Changeable later?
Partitioning	Low (date, region)	No — requires full rewrite
Z-ORDER	Medium to high	Yes, via a new OPTIMIZE
Liquid clustering	Any, including high	Yes, without rewriting existing data

11. VACUUM, time travel and retention

Because updates write new files and tombstone old ones, storage grows until VACUUM removes files no longer referenced and older than the retention threshold.

```
-- What would be deleted (always dry-run first)
VACUUM portfolio.ecom_silver.orders RETAIN 168 HOURS DRY RUN;

VACUUM portfolio.ecom_silver.orders RETAIN 168 HOURS;    -- 7 days

-- Time travel, by version or timestamp
SELECT * FROM portfolio.ecom_silver.orders VERSION AS OF 42;
SELECT * FROM portfolio.ecom_silver.orders TIMESTAMP AS OF '2026-09-01T00:00:00';

-- Roll back a bad load
RESTORE TABLE portfolio.ecom_silver.orders TO VERSION AS OF 41;
```

Do not lower retention below 7 days The default 168-hour threshold protects in-flight readers and long-running streams. Vacuuming more aggressively can delete files a running query still expects and will fail that query. It also destroys the time-travel window you would want during an incident.

12. Maintenance checklist

Task	Cadence	Command
Compact small files	Daily, recent partitions	OPTIMIZE ... WHERE date >= ...
Re-cluster hot tables	Weekly	OPTIMIZE ... ZORDER BY (...)
Reclaim storage	Weekly	VACUUM ... RETAIN 168 HOURS
Refresh statistics	After large loads	ANALYZE TABLE ... COMPUTE STATISTICS
Check table health	Weekly	DESCRIBE DETAIL (numFiles, sizeInBytes)
Audit recent operations	On incident	DESCRIBE HISTORY

```
-- A quick health read: numFiles climbing while sizeInBytes is flat
-- means small-file accumulation and an OPTIMIZE is overdue.
DESCRIBE DETAIL portfolio.ecom_silver.orders;
```

Enable auto-maintenance where available `ALTER TABLE ... SET TBLPROPERTIES (delta.autoOptimize.optimizeWrite = true, delta.autoOptimize.autoCompact = true)` handles most small-file pressure without a scheduled job.